PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Ariel Inácio Jordão Coelho
Lucca Mendes Alves Pellegrini

Desenvolvimento de uma Unidade Lógica e Aritmética (ALU) de 8 Bits em VHDL Concorrente

Belo Horizonte

2026

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Ariel Inácio Jordão Coelho
Lucca Mendes Alves Pellegrini

Desenvolvimento de uma Unidade Lógica e Aritmética (ALU) de 8 Bits em VHDL Concorrente

Relatório técnico do Primeiro Trabalho Pratico (TP1), apresentado à disciplina de Sistemas Reconfiguráveis do Curso de Engenharia de Computação da Pontifícia Universidade Católica de Minas Gerais.
Área de concentração: Sistemas Digitais e Reconfiguráveis
Professor: Francisco Manoel Pinto Garcia

Belo Horizonte

2026

Resumo

Este relatório apresenta o desenvolvimento de uma Unidade Lógica e Aritmética (ALU) de 8 bits, totalmente combinacional, descrita em VHDL com código exclusivamente concorrente e sintetizada no ambiente Quartus II 9.1sp2. A ALU executa 16 operações (lógicas, aritméticas, de deslocamento e de rotação) selecionadas por uma palavra de 4 bits e gera cinco flags de status: negativo, overflow, zero, half carry/borrow e carry/borrow. São descritos a especificação, a arquitetura adotada (um somador/subtrator unificado compartilhado pelas quatro operações aritméticas), o código comentado, os casos de teste e os resultados de simulação funcional, que concordam com os valores esperados.

Palavras-chave: ALU. VHDL. Lógica combinacional. Sistemas reconfiguráveis. Quartus.

Abstract

This report presents the design of an 8-bit, fully combinational Arithmetic Logic Unit (ALU) described in VHDL using concurrent code only and targeted at Quartus II 9.1sp2. The ALU performs 16 operations (logic, arithmetic, shift and rotate) chosen by a 4-bit select word and produces five status flags: negative, overflow, zero, half carry/borrow and carry/borrow. The specification, the architecture (a single adder/subtractor shared by the four arithmetic operations), the commented source code, the test cases and the functional simulation results are described; the results match the expected values.

Keywords: ALU. VHDL. Combinational logic. Reconfigurable systems. Quartus.

Lista de Figuras

Figura 1 – Diagrama de blocos da ALU. 15
Figura 2 – Operações de deslocamento e rotação. 17
Figura 3 – Implementação sintetizada da ALU em portas lógicas, somadores e multiplexadores. 19
Figura 4 – Simulação funcional da operação PASS_B (`op_sel = 0000`). 29
Figura 5 – Simulação funcional da operação OR (`op_sel = 0001`). 29
Figura 6 – Simulação funcional da operação XOR (`op_sel = 0010`). 29
Figura 7 – Simulação funcional da operação AND (`op_sel = 0011`). 29
Figura 8 – Simulação funcional da operação ADDC (`op_sel = 0100`). 29
Figura 9 – Simulação funcional da operação ADD (`op_sel = 0101`). 30
Figura 10 – Simulação funcional da operação SUBC (`op_sel = 0110`). 30
Figura 11 – Simulação funcional da operação SUB (`op_sel = 0111`). 30
Figura 12 – Simulação funcional da operação NOT (`op_sel = 1000`). 30
Figura 13 – Simulação funcional da operação SRA (`op_sel = 1001`). 30
Figura 14 – Simulação funcional da operação SRL (`op_sel = 1010`). 30
Figura 15 – Simulação funcional da operação SLL (`op_sel = 1011`). 31
Figura 16 – Simulação funcional da operação RRC (`op_sel = 1100`). 31
Figura 17 – Simulação funcional da operação RLC (`op_sel = 1101`). 31
Figura 18 – Simulação funcional da operação RR (`op_sel = 1110`). 31
Figura 19 – Simulação funcional da operação RL (`op_sel = 1111`). 31

Lista de Tabelas

Tabela 1 – Carry de entrada do somador em cada operação aritmética. 13
Tabela 2 – Operações da ALU. 16
Tabela 3 – Casos de teste da operação PASS_B. 23
Tabela 4 – Casos de teste da operação OR. 23
Tabela 5 – Casos de teste da operação XOR. 24
Tabela 6 – Casos de teste da operação AND. 24
Tabela 7 – Casos de teste da operação ADDC. 24
Tabela 8 – Casos de teste da operação ADD. 25
Tabela 9 – Casos de teste da operação SUBC. 25
Tabela 10 – Casos de teste da operação SUB. 25
Tabela 11 – Casos de teste da operação NOT. 26
Tabela 12 – Casos de teste da operação SRA. 26
Tabela 13 – Casos de teste da operação SRL. 26
Tabela 14 – Casos de teste da operação SLL. 27
Tabela 15 – Casos de teste da operação RRC. 27
Tabela 16 – Casos de teste da operação RLC. 27
Tabela 17 – Casos de teste da operação RR. 28
Tabela 18 – Casos de teste da operação RL. 28

Lista de abreviaturas e siglas

ALU	*Arithmetic Logic Unit* (unidade lógica e aritmética)
ASIC	*Application-Specific Integrated Circuit*
EDA	*Electronic Design Automation*
FPGA	*Field-Programmable Gate Array*
HDL	*Hardware Description Language*
IEEE	*Institute of Electrical and Electronics Engineers*
LE	*Logic Element*
LUT	*Look-Up Table*
MSI	*Medium Scale Integration*
PLD	*Programmable Logic Device*
RTL	*Register Transfer Level*
VHDL	*VHSIC Hardware Description Language*
VHSIC	*Very High Speed Integrated Circuit*

Sumário

1 Introdução 9

1.1 Objetivos 9

1.2 Ferramentas e restrições 9

1.3 Organização do relatório 10

2 Fundamentação teórica 11

2.1 Linguagens de descrição de hardware e VHDL 11

2.2 Estrutura de um projeto em VHDL 11

2.3 Representação em complemento de dois 12

2.4 Soma e subtração 12

2.5 Flags de status 13

2.6 Operações aritméticas com std_logic_vector 13

3 Especificação da ALU 15

3.1 Entrada e Saida 15

3.2 Operações 16

4 Implementação em VHDL 18

4.1 Arquitetura 18

4.2 Decisões de projeto 19

4.3 Código-fonte 20

5 Simulação e Resultados 23

5.1 Metodologia 23

5.2 Casos de teste e resultados esperados 23

5.3 Formas de ondas 29

5.4 Análise dos resultados 32

5.4.1 Operações lógicas 32

5.4.2 Operações de soma 32

5.4.3 Operações de subtração 33

5.4.4 Deslocamentos e rotações 33

5.4.5 Avaliação geral das flags 34

6 Conclusão 35

Referências **36**
APÊNDICE A Arquivos do projeto **37**

1 Introdução

A Unidade Lógica e Aritmética (ALU) é o bloco do processador responsável por executar as operações sobre os dados. Neste trabalho, desenvolvido na disciplina de Sistemas Reconfiguráveis, projeta-se uma ALU de 8 bits em VHDL (Ashenden, 2008; Pedroni, 2010), que recebe dois operandos, um carry de entrada e um código de operação, e devolve o resultado e um conjunto de flags de status.

1.1 Objetivos

O objetivo geral é descrever, simular e documentar uma ALU combinacional de 8 bits com 16 funções. Os objetivos específicos são:

- descrever a ALU em VHDL usando **exclusivamente código concorrente**, com entradas e saídas dos tipos `std_logic` e `std_logic_vector`;
- garantir que o circuito seja puramente combinacional, sem *latches* nem *flip-flops*;
- gerar corretamente as flags de negativo, overflow, zero, *half carry/borrow* e *carry/borrow*;
- definir casos de teste, simular o projeto e comparar os resultados com os valores esperados.

1.2 Ferramentas e restrições

O projeto foi desenvolvido no Quartus II versão 9.1sp2, conforme exigido na especificação. Todo o código obedece às restrições abaixo.

- Não há comandos `process`; todas as atribuições são concorrentes (`<=`, `with-select`, `when-else`).
- Os pacotes `std_logic_1164` e `numeric_std` são usados; os tipos `unsigned` aparecem apenas como sinais internos do somador.
- mplementar um circuito totalmente combinacional, ou seja, sem latches nem flip-flops.

1.3 Organização do relatório

O Capítulo 2 resume os conceitos necessários. O Capítulo 3 detalha a especificação da ALU. O Capítulo 4 apresenta a arquitetura e o código VHDL. O Capítulo 5 descreve os casos de teste e os resultados. Por fim, a Conclusão sintetiza o trabalho.

O Capítulo 2 resume os conceitos de VHDL e de projeto combinacional apresentados nas aulas e utilizados no projeto. O Capítulo 3 apresenta a especificação da ALU. O Capítulo 4 descreve a implementação em VHDL e o Capítulo 5 apresenta a simulação, os casos de teste e os resultados. Por fim, o Capítulo 6 reúne as conclusões.

2 Fundamentação teórica

2.1 Linguagens de descrição de hardware e VHDL

Uma linguagem de descrição de hardware (HDL) é usada para a descrição formal e o projeto de circuitos eletrônicos, sobretudo digitais. Ela permite descrever o funcionamento, a concepção e a organização de um circuito e testá-lo por meio de simulação (GARCIA; MAGALHÃES, 2024). Dentre as vantagens de uma HDL padronizada estão a independência do fornecedor, a porta- bilidade e a reusabilidade. Em relação ao diagrama esquemático, a descrição por HDL oferece projeto independente da tecnologia, facilidade de atualização, redução do tempo e do custo de projeto e redução do volume de documentação (GARCIA; MAGALHÃES, 2024).

2.2 Estrutura de um projeto em VHDL

Um projeto VHDL possui três unidades fundamentais (Garcia; Magalhães, 2024):

- `LIBRARY`: lista os pacotes e bibliotecas usados no projeto;
- `ENTITY`: especifica as entradas e saídas do circuito, com o nome do arquivo `.vhd` igual ao nome da entidade;
- `ARCHITECTURE`: contém o código que descreve o funcionamento do circuito.

Os modos das portas são `IN`, `OUT`, `BUFFER` e `INOUT`. Uma porta `OUT` não pode ser lida dentro da arquitetura, ao contrário de uma porta `BUFFER` (Garcia; Magalhães, 2024). A atribuição a um `SIGNAL` ou a uma porta usa o operador <=; os comentários começam com − e vão até o fim da linha. A linguagem não distingue maiúsculas de minúsculas, mas é recomendável escrever as palavras reservadas em maiúsculas e os nomes de sinais em minúsculas (Garcia; Magalhães, 2024).

O Código 2.1 reproduz o exemplo de código completo apresentado na aula 3.

Listing 2.1 – Exemplo de ENTITY e ARCHITECTURE (aula 3).

```
1  ENTITY Cap3ex1 IS
2      PORT (
3          a      : IN BIT;
4          b      : IN BIT;
5          x,y,z  : OUT BIT
6      );
7  END Cap3ex1;
8  ------------------------------
9  ARCHITECTURE arch OF Cap3ex1 IS
10 BEGIN
11     x <= a AND b;    -- saida x
12     y <= a OR b;     -- saida y
13     z <= a XOR b;    -- saida z
14 END arch;
```

Fonte: Codigo retirado do slide aula 6 de Garcia.

2.3 Representação em complemento de dois

Em palavras de 8 bits em complemento de dois, o bit 7 é o bit de sinal (Harris; Harris, 2012): valores de -128 a 127 são representáveis, e o bit 7 igual a '1' indica número negativo. Por isso a flag de negativo (N) é simplesmente o bit 7 do resultado.

2.4 Soma e subtração

A soma bit a bit com propagação de carry é feita por um somador binário. A subtração pode ser reduzida a uma soma, pois

$$a - b - c = a + \overline{b} + \overline{c}, \tag{2.1}$$

onde $\overline{b}$ é o complemento bit a bit de b e $\overline{c}$ é o inverso do carry de entrada. Isso permite que um único somador atenda às quatro operações aritméticas: basta inverter b nas subtrações e escolher adequadamente o carry de entrada do somador (Tabela 1).

Tabela 1 – Carry de entrada do somador em cada operação aritmética.

Operação	Entrada B do somador	Carry de entrada do somador	Resultado
ADDC	b	c_{in}	$a + b + c_{in}$
ADD	b	0	$a + b$
SUBC	$\overline{b}$	$\overline{c_{in}}$	$a - b - c_{in}$
SUB	$\overline{b}$	1	$a - b$

Fonte: Autoria propria.

2.5 Flags de status

Carry/borrow (C) Na soma, é o carry do bit 7. Na subtração, é o *borrow* (empréstimo), igual ao carry do somador invertido. Nas rotações e deslocamentos guarda o bit que sai da palavra.

Half carry/borrow (H) Análogo ao C, mas referente à passagem do bit 3 para o bit 4.

Zero (Z) Vale '1' quando todos os bits do resultado são zero.

Negative (N) Cópia do bit 7 do resultado.

Overflow (V) Indica que o resultado em complemento de dois não cabe em 8 bits. Com b_{op} sendo o operando efetivamente somado (b ou $\overline{b}$), ocorre overflow quando os sinais de a e b_{op} são iguais e o sinal do resultado é diferente:

$$V = \overline{a_7 \oplus b_{op,7}} \cdot (a_7 \oplus r_7). \tag{2.2}$$

2.6 Operações aritméticas com std_logic_vector

O tipo `std_logic_vector` do pacote IEEE 1164 só permite operações lógicas. Para realizar operações aritméticas há duas soluções (Garcia; Magalhães, 2025):

- usar um dos pacotes `std_logic_unsigned` ou `std_logic_signed` (nunca os dois ao mesmo tempo);
- usar o pacote `numeric_std`, que define os tipos `signed` e `unsigned` e funções de conversão como `to_integer`, `to_unsigned` e `to_signed`. Esse pacote deve ser preferido ao pacote `arith`.

Para detectar o *carry* (“vai um”) no bit mais significativo, a operação deve ser feita com 9 bits, e o bit de *carry* é o bit mais significativo do resultado (Garcia; Magalhães, 2025). O Código 2.2 reproduz o exemplo `alu_3` da aula 6, que implementa uma ALU de quatro operações com saída de *carry*, e serve de ponto de partida para o projeto.

Listing 2.2 – Arquitetura do exemplo alu_3 (aula 6).

```
ARCHITECTURE arch OF alu_3 IS
    SIGNAL aux : STD_LOGIC_VECTOR(8 DOWNTO 0);
BEGIN
    WITH op SELECT
        aux <= '0' & (a AND b)         WHEN "00",
               '0' & NOT a             WHEN "01",
               ('0' & a) + ('0' & b)   WHEN "10",
               ('0' & a) - ('0' & b)   WHEN OTHERS;

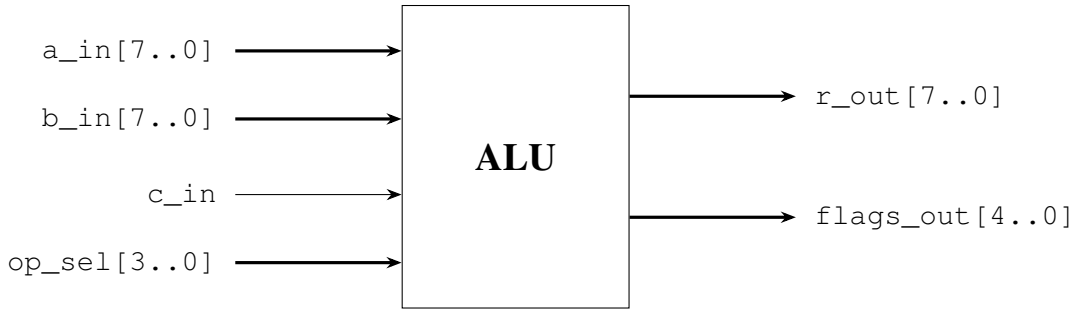
    r <= aux(7 DOWNTO 0);
    c <= aux(8);
    z <= '1' WHEN aux(7 DOWNTO 0) = "00000000" ELSE '0';
END arch;
```

Fonte: Codigo retirado do slide aula 6 de Garcia.

3 Especificação da ALU

A ALU realiza operações lógicas e aritméticas em palavras de 8 bits e executa 16 funções diferentes, entre operações lógicas, aritméticas, de rotação e de deslocamento. O circuito é totalmente combinacional (Garcia; Magalhães, 2026). A Figura 1 mostra o diagrama de blocos com as entradas e saídas.

Figura 1 – Diagrama de blocos da ALU.



Fonte: Elaborada pelo autor, com base em Garcia e Magalhães (2026).

3.1 Entrada e Saida

Quadro 1 – Portas da ALU.

Porta	Direção	Descrição
`a_in[7..0]`	entrada	Operando A
`b_in[7..0]`	entrada	Operando B (operações com dois operandos)
`c_in`	entrada	Carry de entrada (somas, subtrações com carry e rotações)
`op_sel[3..0]`	entrada	Seleção da operação
`r_out[7..0]`	saída	Resultado
`flags_out[4..0]`	saída	Flags: [0] N, [1] V, [2] Z, [3] H, [4] C

Fonte: Especificação do primeiro projeto (TP1).

Quadro 2 – *Flags* de saída da ALU.

Bit	**Nome**	**Significado**
`flags_out[0]`	*negative* (`n_flag`)	Bit 7 do resultado. Nas operações aritméticas indica resultado positivo (`'0'`) ou negativo (`'1'`).
`flags_out[1]`	*overflow* (`v_flag`)	Vale `'1'` quando há *overflow* em soma ou subtração.
`flags_out[2]`	*zero* (`z_flag`)	Vale `'1'` quando o resultado é zero.
`flags_out[3]`	*half carry/borrow* (`h_flag`)	Na soma, é o *carry out* do bit 3 para o bit 4; na subtração, é o *borrow out*.
`flags_out[4]`	*carry/borrow* (`c_flag`)	Na soma, é o *carry out* do bit mais significativo; na subtração, é o *borrow out*. Também é usada nas operações de rotação.

Fonte: Adaptado de Garcia e Magalhães (2026).

3.2 Operações

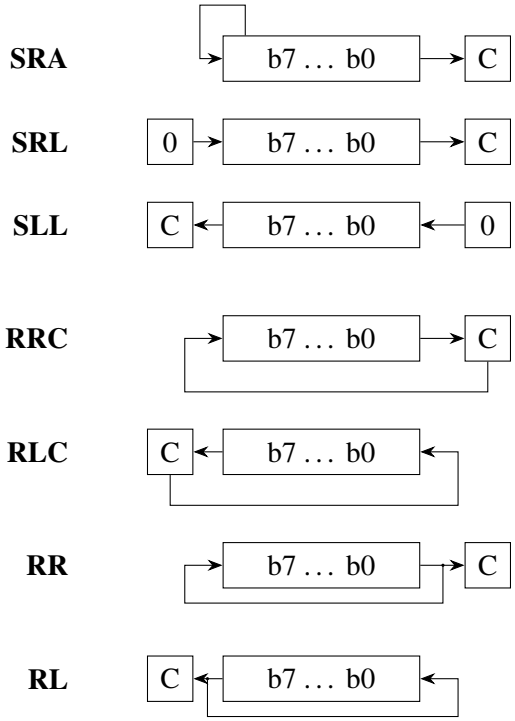
Tabela 2 – Operações da ALU.

`op_sel`	Mnemônico	Operação	Flags C / H / V
0000	PASS_B	$r = b$	0 / 0 / –
0001	OR	$r = a \vee b$	0 / 0 / –
0010	XOR	$r = a \oplus b$	0 / 0 / –
0011	AND	$r = a \wedge b$	0 / 0 / –
0100	ADDC	$r = a + b + c_{in}$	carry / half carry / ovf
0101	ADD	$r = a + b$	carry / half carry / ovf
0110	SUBC	$r = a - b - c_{in}$	borrow / half borrow / ovf
0111	SUB	$r = a - b$	borrow / half borrow / ovf
1000	NOT	$r = \overline{a}$	0 / – / –
1001	SRA	$r = a_7, a_{7..1}$ (desloc. aritmético à direita)	a_0 / – / –
1010	SRL	$r = 0, a_{7..1}$ (desloc. lógico à direita)	a_0 / – / –
1011	SLL	$r = a_{6..0}, 0$ (desloc. lógico à esquerda)	a_7 / – / –
1100	RRC	$r = c_{in}, a_{7..1}$ (rotação à direita pelo carry)	a_0 / – / –
1101	RLC	$r = a_{6..0}, c_{in}$ (rotação à esquerda pelo carry)	a_7 / – / –
1110	RR	$r = a_0, a_{7..1}$ (rotação à direita)	a_0 / – / –
1111	RL	$r = a_{6..0}, a_7$ (rotação à esquerda)	a_7 / – / –

Em todas as operações, Z = ‘1’ se $r = 0$ e N = r_7. O símbolo “–” indica *don’t care*.

As operações de deslocamento e rotação são ilustradas na Figura 2, em que **C** é o *carry*.

Figura 2 – Operações de deslocamento e rotação.



Fonte: Adaptado de Garcia e Magalhães (2026).

4 Implementação em VHDL

4.1 Arquitetura

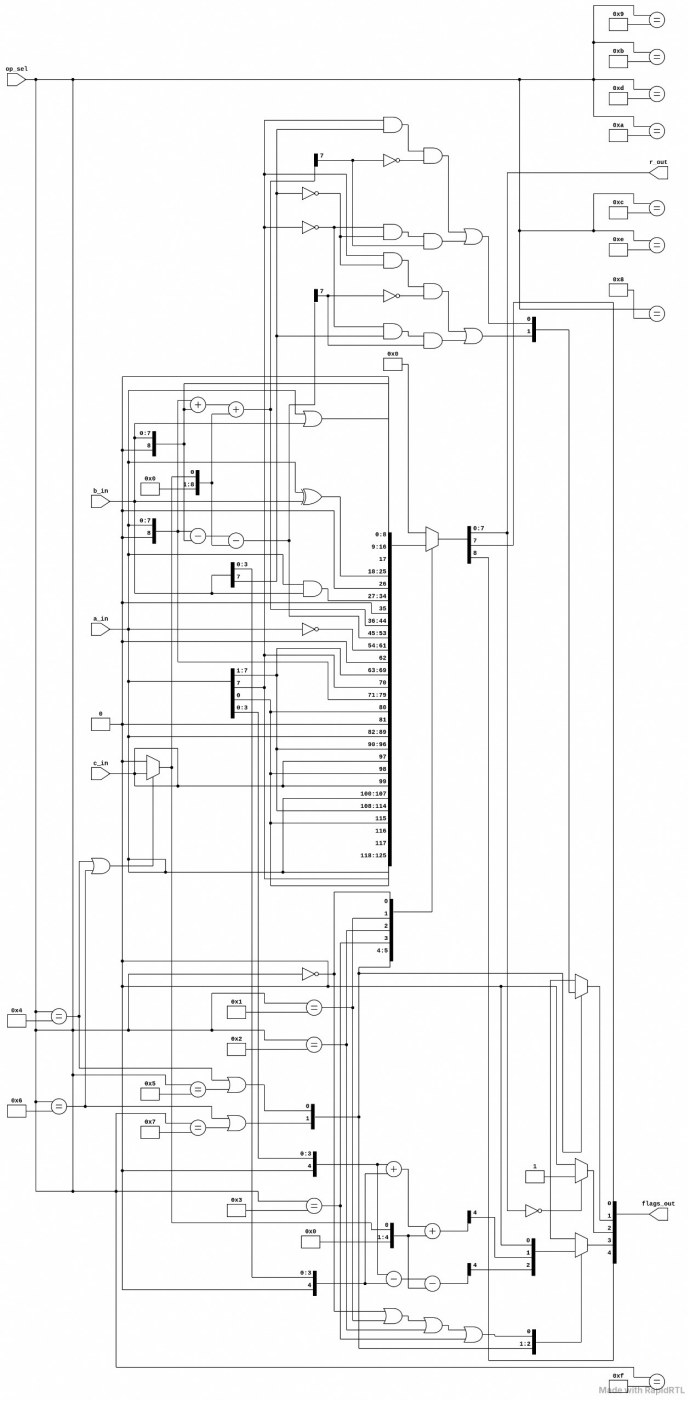
A ALU pode ser descrita funcionalmente em três blocos principais: o caminho aritmético, a seleção do resultado e a geração das flags. Essa divisão representa uma visão conceitual da arquitetura, enquanto a implementação sintetizada apresentada na Figura 3 detalha as portas lógicas, os somadores e as redes de multiplexadores utilizados pelo Quartus.

1. **Caminho aritmético**: implementa um somador/subtrator de 8 bits compartilhado pelas operações ADDC, ADD, SUBC e SUB. O caminho é dividido em dois nibbles de quatro bits. O carry produzido pelo nibble inferior é encaminhado ao nibble superior e utilizado para determinar a flag de *half carry* ou *half borrow*. Nas subtrações, o operando B é invertido e o carry inicial é ajustado para produzir a subtração por complemento de dois;

2. **Seleção do resultado**: combina os resultados das operações lógicas, aritméticas, de deslocamento e de rotação. A seleção é controlada por `op_sel`. No código VHDL, essa função é descrita por uma atribuição `with-select`; durante a síntese, ela é implementada como uma rede de multiplexadores, conforme observado na Figura 3;

3. **Geração das flags**: calcula as flags a partir do resultado selecionado e da operação executada. A flag N é o bit mais significativo de `r_out`, enquanto Z indica resultado nulo. A flag C recebe o carry ou borrow das operações aritméticas e o bit deslocado nas operações de deslocamento e rotação. As flags H e V são geradas pelo caminho aritmético e, nas operações em que não são definidas pela especificação, são fixadas em zero.

Figura 3 – Implementação sintetizada da ALU em portas lógicas, somadores e multiplexadores.



Fonte: Elaborado a partir do site RapidRTL, utilizado como base o coigo fonte (alu.vhd).

4.2 Decisões de projeto

- **Subtração por soma**: usa-se a identidade da Equação 2.1. O bit `op_sel(1)` distingue subtração (`1`) de soma (`0`) e `op_sel(0)` indica se o carry de entrada é usado (`0`). O carry

de entrada do somador é

$$c_{add} = \overline{s} \cdot \overline{op_0} \cdot c_{in} \; + \; s \cdot \overline{\overline{op_0} \cdot c_{in}},$$

em que s = `op_sel(1)`.

- **Borrow**: em subtrações, C e H são o carry do somador invertido (XOR com s). Equação 2.1.
- **Overflow**: calculado pela Equação 2.2 usando o operando efetivo b_{op}.
- ***Don't care***: nas operações em que a especificação não define H ou V, essas flags são fixadas em '0', o que evita lógica desnecessária e saídas indefinidas na simulação.

4.3 Código-fonte

O Código 4.1 apresenta a entidade e a arquitetura completas, com comentários.

Listing 4.1 – Descrição VHDL da ALU (`alu.vhd`).

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.std_logic_unsigned.all;
4
5  ENTITY alu IS
6      PORT (
7          a_in      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
8          b_in      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
9          c_in      : IN  STD_LOGIC;
10         op_sel    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
11         r_out     : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
12         flags_out : OUT STD_LOGIC_VECTOR(4 DOWNTO 0)
13     );
14 END ENTITY;
15
16 ARCHITECTURE arch OF alu IS
17     -- Códigos das operações -----------------------------------------
18     CONSTANT OP_PASS_B : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0000";
19     CONSTANT OP_OR     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0001";
20     CONSTANT OP_XOR    : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0010";
21     CONSTANT OP_AND    : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0011";
22     CONSTANT OP_ADDC   : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0100";
23     CONSTANT OP_ADD    : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0101";
24     CONSTANT OP_SUBC   : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0110";
25     CONSTANT OP_SUB    : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0111";
26     CONSTANT OP_NOT    : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1000";
```

```
27      CONSTANT OP_SRA     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1001";
28      CONSTANT OP_SRL     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1010";
29      CONSTANT OP_SLL     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1011";
30      CONSTANT OP_RRC     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1100";
31      CONSTANT OP_RLC     : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1101";
32      CONSTANT OP_RR      : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1110";
33      CONSTANT OP_RL      : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1111";
34
35      -- Carry-in efetivo (so vale c_in em ADDC e SUBC)
36      SIGNAL cin_ef : STD_LOGIC;
37
38      -- Somador e subtrator (bit 8 é o carry)
39      SIGNAL soma : STD_LOGIC_VECTOR(8 DOWNTO 0);
40      SIGNAL dif  : STD_LOGIC_VECTOR(8 DOWNTO 0);
41
42      -- Somador e subtrator dos nibbles baixos (bit 4 é o half-carry)
43      SIGNAL h_soma : STD_LOGIC_VECTOR(4 DOWNTO 0);
44      SIGNAL h_dif  : STD_LOGIC_VECTOR(4 DOWNTO 0);
45
46      -- Overflow de soma/subtração
47      SIGNAL v_soma : STD_LOGIC;
48      SIGNAL v_dif  : STD_LOGIC;
49
50      -- Resultado
51      SIGNAL res   : STD_LOGIC_VECTOR(8 DOWNTO 0);
52      SIGNAL r_int : STD_LOGIC_VECTOR(7 DOWNTO 0); -- resultado sem o carry
53
54      SIGNAL n_flag, v_flag, z_flag, h_flag, c_flag : STD_LOGIC;
55  BEGIN
56      -- Seleção da operação -------------------------------------------------
57      WITH op_sel SELECT
58          res <= '0'     & b_in                          WHEN OP_PASS_B,
59                 '0'     & (a_in OR  b_in)               WHEN OP_OR,
60                 '0'     & (a_in XOR b_in)               WHEN OP_XOR,
61                 '0'     & (a_in AND b_in)               WHEN OP_AND,
62                 soma                                    WHEN OP_ADDC | OP_ADD,
63                 dif                                     WHEN OP_SUBC | OP_SUB,
64                 '0'     & (NOT a_in)                    WHEN OP_NOT,
65                 a_in(0) & a_in(7) & a_in(7 DOWNTO 1)    WHEN OP_SRA,
66                 a_in(0) & '0' & a_in(7 DOWNTO 1)        WHEN OP_SRL,
67                 a_in(7) & a_in(6 DOWNTO 0) & '0'        WHEN OP_SLL,
68                 a_in(0) & c_in & a_in(7 DOWNTO 1)       WHEN OP_RRC,
69                 a_in(7) & a_in(6 DOWNTO 0) & c_in       WHEN OP_RLC,
70                 a_in(0) & a_in(0) & a_in(7 DOWNTO 1)    WHEN OP_RR,
71                 a_in(7) & a_in(6 DOWNTO 0) & a_in(7)    WHEN OP_RL,
72                 (OTHERS => '0')                         WHEN OTHERS;
73
```

```
74      r_int <= res(7 DOWNTO 0);
75      r_out <= r_int;
76
77      -- Unidade aritmética ------------------------------------------------
78      WITH op_sel SELECT
79          cin_ef <= c_in WHEN OP_ADDC | OP_SUBC,
80                    '0'  WHEN OTHERS;
81
82      soma <= ('0' & a_in) + ('0' & b_in) + cin_ef;
83      dif  <= ('0' & a_in) - ('0' & b_in) - cin_ef;
84
85      h_soma <= ('0' & a_in(3 DOWNTO 0)) + ('0' & b_in(3 DOWNTO 0)) + cin_ef;
86      h_dif  <= ('0' & a_in(3 DOWNTO 0)) - ('0' & b_in(3 DOWNTO 0)) - cin_ef;
87
88      -- Soma: overflow se os operandos têm o mesmo sinal e o resultado tem
89      -- sinal diferente.
90      v_soma <= (a_in(7) AND b_in(7) AND (NOT soma(7))) OR
91                ((NOT a_in(7)) AND (NOT b_in(7)) AND soma(7));
92
93      -- Subtração: overflow se os operandos têm sinais diferentes e o
94      -- resultado tem o sinal de b.
95      v_dif  <= (a_in(7) AND (NOT b_in(7)) AND (NOT dif(7))) OR
96                ((NOT a_in(7)) AND b_in(7) AND dif(7));
97
98      -- Flags -------------------------------------------------------------
99      n_flag <= res(7);
100     c_flag <= res(8);
101
102     WITH r_int SELECT
103         z_flag <= '1' WHEN "00000000",
104                   '0' WHEN OTHERS;
105
106     WITH op_sel SELECT
107         h_flag <= '0'        WHEN OP_PASS_B | OP_OR | OP_XOR | OP_AND,
108                   h_soma(4)  WHEN OP_ADDC | OP_ADD,
109                   h_dif(4)   WHEN OP_SUBC | OP_SUB,
110                   '-'        WHEN OTHERS;
111
112     WITH op_sel SELECT
113         v_flag <= v_soma     WHEN OP_ADDC | OP_ADD,
114                   v_dif      WHEN OP_SUBC | OP_SUB,
115                   '-'        WHEN OTHERS;
116
117     flags_out <= c_flag & h_flag & z_flag & v_flag & n_flag;
118 END ARCHITECTURE;
```

Fonte: Autoria propria.

5 Simulação e Resultados

5.1 Metodologia

A simulação funcional foi feita no Quartus II 9.1sp2 com o editor de formas de onda (arquivo `.vwf`). Como o circuito é combinacional, cada vetor de teste define `a_in`, `b_in`, `c_in` e `op_sel`, e o resultado é conferido após o atraso de propagação.

Os casos de teste cobrem: (i) todas as 16 operações; (ii) os seis exemplos de ADD da especificação, assim como 6 outros exemplos para as demais 15 operações; (iii) situações-limite das flags, como carry, *half carry*, overflow positivo e negativo, resultado zero e resultado negativo; e (iv) o uso de `c_in` em ADDC, SUBC, RRC e RLC.

5.2 Casos de teste e resultados esperados

As Tabelas 3 a 18 lista os casos usados. Os traços indicam flags *don't care* (fixadas em '0' no projeto).

Tabela 3 – Casos de teste da operação PASS_B.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
1	FF	A5	0	A5	0	0	0	–	1
2	FF	00	1	00	0	0	1	–	0
3	12	7F	0	7F	0	0	0	–	0
4	55	80	1	80	0	0	0	–	1
5	00	FF	0	FF	0	0	0	–	1
6	F0	01	1	01	0	0	0	–	0

Fonte: Autoria propria.

Tabela 4 – Casos de teste da operação OR.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
7	00	00	0	00	0	0	1	–	0
8	F0	0F	0	FF	0	0	0	–	1

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
9	A5	00	0	A5	0	0	0	–	1
10	80	01	0	81	0	0	0	–	1
11	55	AA	0	FF	0	0	0	–	1
12	0F	30	1	3F	0	0	0	–	0

Fonte: Autoria propria.

Tabela 5 – Casos de teste da operação XOR.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
13	FF	FF	0	00	0	0	1	–	0
14	AA	55	0	FF	0	0	0	–	1
15	F0	0F	0	FF	0	0	0	–	1
16	B5	4C	0	F9	0	0	0	–	1
17	00	00	0	00	0	0	1	–	0
18	80	80	1	00	0	0	1	–	0

Fonte: Autoria propria.

Tabela 6 – Casos de teste da operação AND.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
19	0F	F0	0	00	0	0	1	–	0
20	FF	FF	0	FF	0	0	0	–	1
21	F0	3C	0	30	0	0	0	–	0
22	B5	4C	0	04	0	0	0	–	0
23	80	FF	0	80	0	0	0	–	1
24	55	AA	1	00	0	0	1	–	0

Fonte: Autoria propria.

Tabela 7 – Casos de teste da operação ADDC.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
25	0F	01	0	10	0	1	0	0	0
26	0F	01	1	11	0	1	0	0	0

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
27	FF	00	1	00	1	1	1	0	0
28	7F	00	1	80	0	1	0	1	1
29	FF	FF	1	FF	1	1	0	0	1
30	80	80	1	01	1	0	0	1	0

Fonte: Autoria propria.

Tabela 8 – Casos de teste da operação ADD.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
31	37	10	0	47	0	0	0	0	0
32	37	19	0	50	0	1	0	0	0
33	77	70	0	E7	0	0	0	1	1
34	77	89	0	00	1	1	1	0	0
35	80	80	0	00	1	0	1	1	0
36	C7	D0	0	97	1	0	0	0	1

Fonte: Autoria propria.

Tabela 9 – Casos de teste da operação SUBC.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
37	10	0F	1	00	0	1	1	0	0
38	10	0F	0	01	0	1	0	0	0
39	00	00	1	FF	1	1	0	0	1
40	80	00	1	7F	0	1	0	1	0
41	50	B0	0	A0	1	0	0	1	1
42	05	05	0	00	0	0	1	0	0

Fonte: Autoria propria.

Tabela 10 – Casos de teste da operação SUB.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
43	10	01	0	0F	0	1	0	0	0
44	05	05	0	00	0	0	1	0	0

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
45	00	01	0	FF	1	1	0	0	1
46	80	01	0	7F	0	1	0	1	0
47	50	B0	0	A0	1	0	0	1	1
48	10	01	1	0F	0	1	0	0	0

Fonte: Autoria propria.

Tabela 11 – Casos de teste da operação NOT.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
49	0F	00	0	F0	0	–	0	–	1
50	FF	00	0	00	0	–	1	–	0
51	00	00	0	FF	0	–	0	–	1
52	AA	55	1	55	0	–	0	–	0
53	80	00	0	7F	0	–	0	–	0
54	7F	00	0	80	0	–	0	–	1

Fonte: Autoria propria.

Tabela 12 – Casos de teste da operação SRA.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
55	81	00	0	C0	1	–	0	–	1
56	01	00	0	00	1	–	1	–	0
57	7E	00	0	3F	0	–	0	–	0
58	80	00	0	C0	0	–	0	–	1
59	FF	00	0	FF	1	–	0	–	1
60	00	00	1	00	0	–	1	–	0

Fonte: Autoria propria.

Tabela 13 – Casos de teste da operação SRL.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
61	81	00	0	40	1	–	0	–	0
62	01	00	0	00	1	–	1	–	0

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
63	FE	00	0	7F	0	–	0	–	0
64	FF	00	0	7F	1	–	0	–	0
65	80	00	0	40	0	–	0	–	0
66	00	00	1	00	0	–	1	–	0

Fonte: Autoria propria.

Tabela 14 – Casos de teste da operação SLL.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
67	81	00	0	02	1	–	0	–	0
68	40	00	0	80	0	–	0	–	1
69	01	00	0	02	0	–	0	–	0
70	FF	00	0	FE	1	–	0	–	1
71	80	00	0	00	1	–	1	–	0
72	00	00	1	00	0	–	1	–	0

Fonte: Autoria propria.

Tabela 15 – Casos de teste da operação RRC.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
73	01	00	1	80	1	–	0	–	1
74	01	00	0	00	1	–	1	–	0
75	00	00	1	80	0	–	0	–	1
76	FF	00	0	7F	1	–	0	–	0
77	80	00	1	C0	0	–	0	–	1
78	A5	00	0	52	1	–	0	–	0

Fonte: Autoria propria.

Tabela 16 – Casos de teste da operação RLC.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
79	80	00	1	01	1	–	0	–	0
80	80	00	0	00	1	–	1	–	0

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
81	00	00	1	01	0	–	0	–	0
82	FF	00	0	FE	1	–	0	–	1
83	01	00	1	03	0	–	0	–	0
84	5A	00	0	B4	0	–	0	–	1

Fonte: Autoria propria.

Tabela 17 – Casos de teste da operação RR.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
85	01	00	0	80	1	–	0	–	1
86	02	00	0	01	0	–	0	–	0
87	80	00	1	40	0	–	0	–	0
88	FF	00	0	FF	1	–	0	–	1
89	A5	00	0	D2	1	–	0	–	1
90	00	00	1	00	0	–	1	–	0

Fonte: Autoria propria.

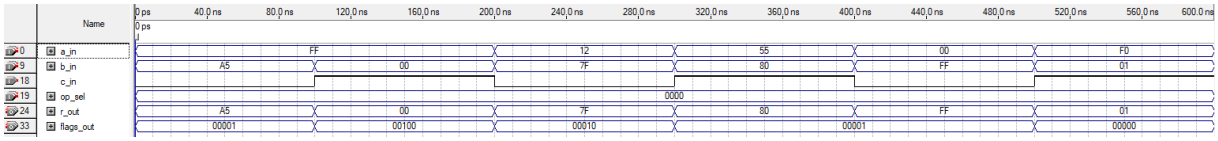
Tabela 18 – Casos de teste da operação RL.

Caso	a_in	b_in	c_in	r_out	C	H	Z	V	N
91	80	00	0	01	1	–	0	–	0
92	40	00	0	80	0	–	0	–	1
93	01	00	1	02	0	–	0	–	0
94	FF	00	0	FF	1	–	0	–	1
95	A5	00	0	4B	1	–	0	–	0
96	00	00	1	00	0	–	1	–	0

Fonte: Autoria propria.

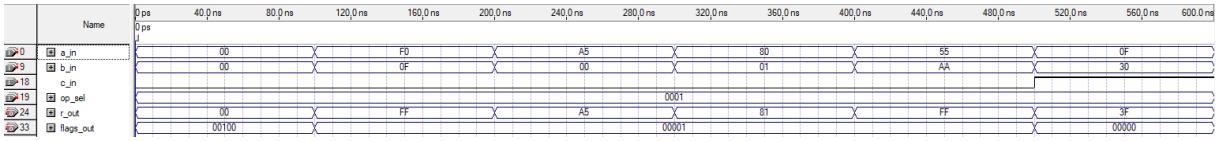
5.3 Formas de ondas

Figura 4 – Simulação funcional da operação PASS_B (`op_sel = 0000`).



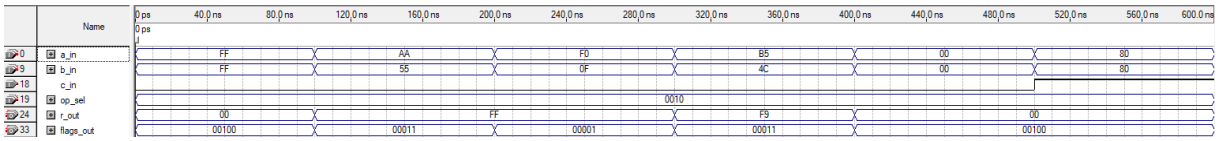
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 5 – Simulação funcional da operação OR (`op_sel = 0001`).



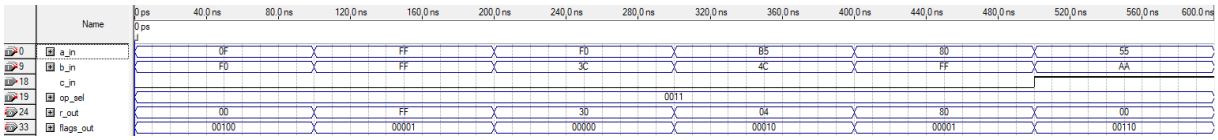
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 6 – Simulação funcional da operação XOR (`op_sel = 0010`).



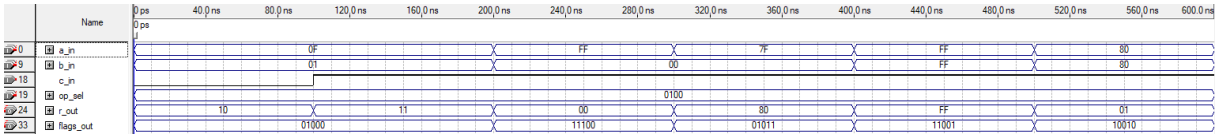
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 7 – Simulação funcional da operação AND (`op_sel = 0011`).



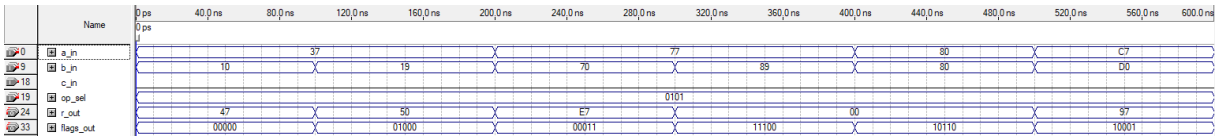
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 8 – Simulação funcional da operação ADDC (`op_sel = 0100`).



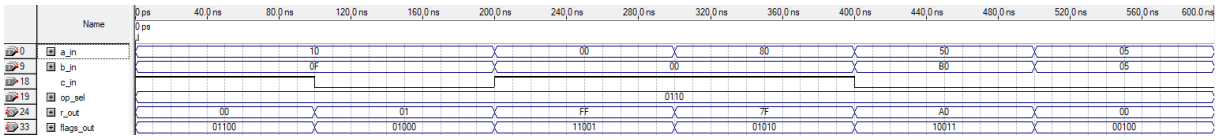
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 9 – Simulação funcional da operação ADD (`op_sel = 0101`).



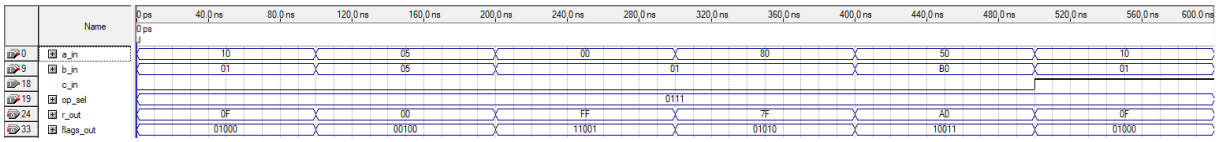
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 10 – Simulação funcional da operação SUBC (`op_sel = 0110`).



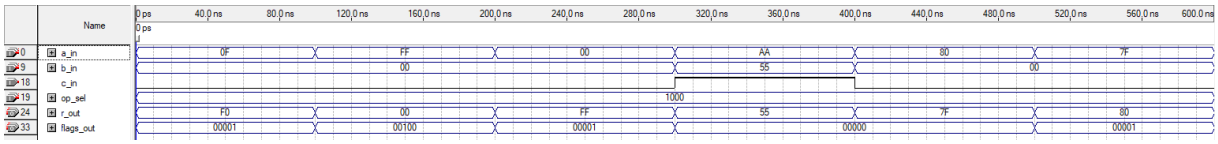
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 11 – Simulação funcional da operação SUB (`op_sel = 0111`).



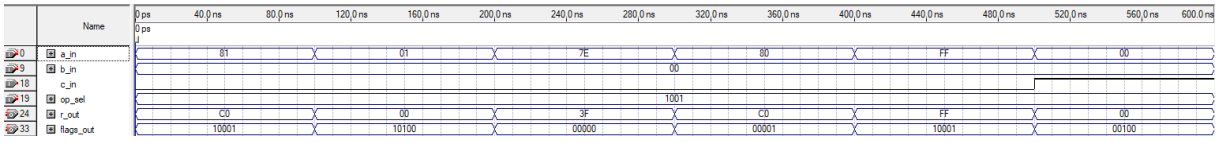
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 12 – Simulação funcional da operação NOT (`op_sel = 1000`).



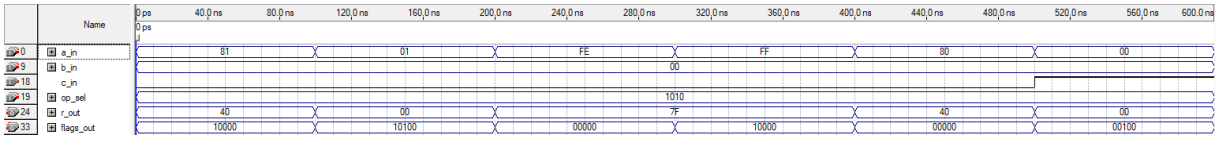
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 13 – Simulação funcional da operação SRA (`op_sel = 1001`).



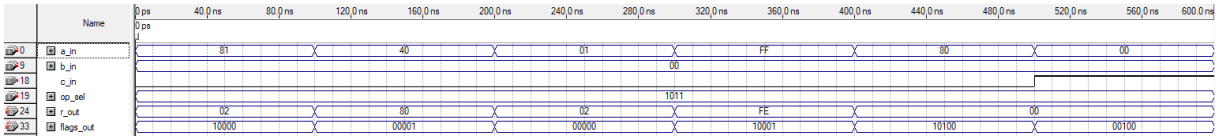
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 14 – Simulação funcional da operação SRL (`op_sel = 1010`).



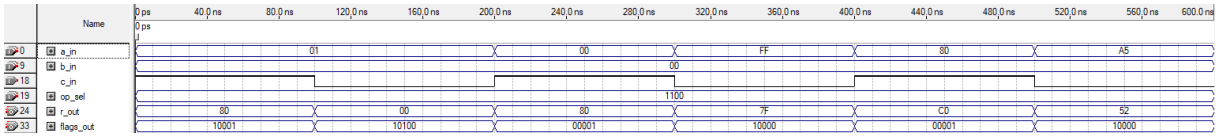
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 15 – Simulação funcional da operação SLL (`op_sel = 1011`).



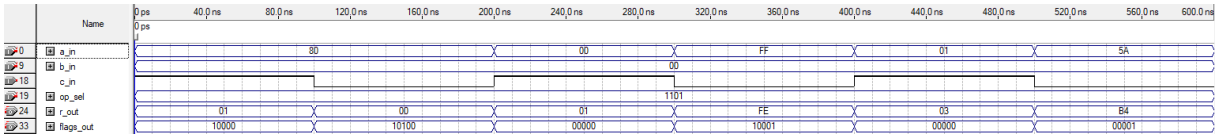
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 16 – Simulação funcional da operação RRC (`op_sel = 1100`).



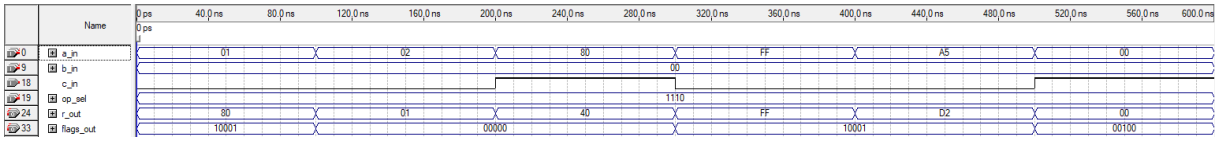
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 17 – Simulação funcional da operação RLC (`op_sel = 1101`).



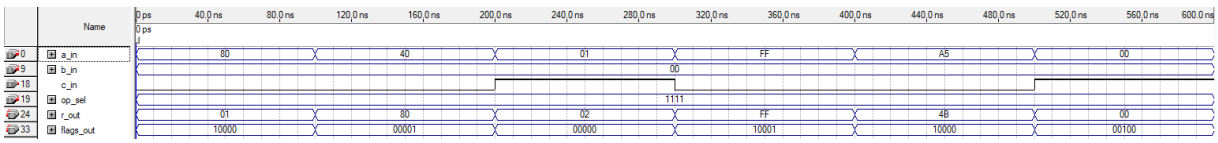
Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 18 – Simulação funcional da operação RR (`op_sel = 1110`).



Fonte: Autoria propria (Quartus II 9.1sp2).

Figura 19 – Simulação funcional da operação RL (`op_sel = 1111`).



Fonte: Autoria propria (Quartus II 9.1sp2).

5.4 Análise dos resultados

Os resultados foram analisados a partir dos 96 casos apresentados nas tabelas individuais de cada operação e das formas de onda obtidas no Quartus II. Em todos os casos, foram comparados o valor de `r_out` e o estado das flags C, H, Z, V e N. Essa comparação é importante porque uma operação pode produzir o resultado numérico correto e, ainda assim, apresentar erro em uma das flags.

5.4.1 Operações lógicas

As operações `PASS_B`, `OR`, `XOR` e `AND` apresentaram o comportamento esperado para todas as combinações testadas. Em `PASS_B`, o resultado acompanha diretamente `b_in`, independentemente do valor de `a_in` e de `c_in`. Nas demais operações, cada bit do resultado corresponde à aplicação da operação lógica entre os bits de mesma posição dos operandos.

Nessas quatro operações, C e H permanecem em zero, pois não existe propagação de carry ou borrow. A flag V também é tratada como "don't care" pela especificação e foi mantida em zero na implementação. As flags Z e N foram geradas a partir do resultado selecionado: Z é ativada somente quando `r_out` é igual a zero, enquanto N reproduz o bit mais significativo do resultado. Os casos com resultados `00` e `FF` confirmam, respectivamente, o funcionamento das flags Z e N.

5.4.2 Operações de soma

As operações `ADDC` e `ADD` confirmaram o funcionamento do caminho aritmético compartilhado. Em `ADDC`, o valor de `c_in` participa da soma; em `ADD`, ele é ignorado e o carry inicial é zero. A diferença pode ser observada nos casos em que os operandos são iguais, mas o carry de entrada varia, como em $0F + 01 + 0 = 10$ e $0F + 01 + 1 = 11$.

Os seis exemplos de `ADD` da especificação também foram reproduzidos. O caso $37h + 10h = 47h$ não produz flags de exceção; $37h + 19h = 50h$ produz *half carry*; e $77h + 70h = E7h$ apresenta overflow e resultado negativo. No caso $77h + 89h = 00h$, ocorre simultaneamente carry, *half carry* e zero. Já $80h + 80h = 00h$ demonstra a ocorrência de carry e overflow com resultado zero, enquanto $C7h + D0h = 97h$ produz carry sem overflow. Esses casos mostram que carry e overflow representam condições diferentes: o primeiro está relacionado ao transporte

além do oitavo bit, e o segundo à interpretação com sinal.

5.4.3 Operações de subtração

Os resultados de `SUBC` e `SUB` confirmam a implementação da subtração por complemento de dois. O operando B é invertido e o carry inicial é ajustado para que a soma produza a diferença desejada. Em `SUBC`, `c_in` é incluído como empréstimo; em `SUB`, a operação é feita sem empréstimo inicial.

Nas subtrações, C e H representam borrow, conforme a convenção adotada no projeto. Por exemplo, $10h - 20h = F0h$ gera borrow e resultado negativo. Também é possível observar a diferença entre borrow e overflow no caso $80h - 01h = 7Fh$: o resultado aritmético excede o intervalo representável com sinal, portanto V é ativada, embora o resultado binário seja positivo. Nos casos em que os operandos são iguais, o resultado é zero, Z é ativada e as flags de borrow permanecem inativas.

5.4.4 Deslocamentos e rotações

Nas operações `SRA`, `SRL` e `SLL`, o bit removido da palavra é copiado para C. A operação `SRA` preserva o bit de sinal ao preencher a posição mais significativa, enquanto `SRL` preenche essa posição com zero. Em `SLL`, o bit menos significativo recebe zero e o bit mais significativo deslocado é utilizado para atualizar C.

Os resultados confirmam, por exemplo, que `81h` deslocado aritmeticamente à direita produz `C0h`, preservando o bit de sinal, ao passo que o deslocamento lógico produz `40h`. Em todas essas operações, H e V não são relevantes e permanecem fixadas em zero, conforme definido na especificação.

Nas rotações com carry, `RRC` e `RLC`, o valor de `c_in` é inserido na palavra durante a rotação. Em `RRC`, o bit menos significativo deslocado ocupa a posição de C e o carry de entrada entra no bit mais significativo. Em `RLC`, ocorre o processo inverso. Os casos com `c_in` igual a zero e a um confirmam que essa entrada não é apenas uma flag de controle, mas participa efetivamente do resultado.

Nas rotações `RR` e `RL`, não há carry de entrada: o bit expulso retorna pela extremidade oposta da palavra. Assim, a rotação de `01h` para a direita produz `80h`, enquanto a rotação de

80h para a esquerda produz 01h. Esses casos também confirmam que C recebe o bit que foi expulso.

5.4.5 Avaliação geral das flags

As flags Z e N foram consistentes em todas as classes de operação. Z foi ativada para resultados 00h, inclusive quando o valor zero foi obtido por uma soma, subtração, operação lógica ou deslocamento. N acompanhou corretamente o bit 7 de r_out, permanecendo ativa para resultados como 80h, A5h, C0h e FFh.

As flags C, H e V foram avaliadas de acordo com a natureza da operação. C e H foram utilizados nas operações aritméticas, C também registrou o bit expulso nos deslocamentos e rotações, e V foi utilizada para indicar overflow com sinal nas somas e subtrações. Nas operações em que a especificação não define uma dessas flags, o valor zero evita estados indefinidos e torna a forma de onda mais simples de interpretar.

De modo geral, a comparação entre as tabelas de resultados esperados e as formas de onda confirma a correção funcional da ALU para as 16 operações implementadas. Os testes também verificam as situações de limite mais relevantes, incluindo carry, borrow, *half carry*, overflow, zero, resultado negativo e o uso do carry de entrada.

6 Conclusão

Foi desenvolvida e verificada uma ALU combinacional de 8 bits com 16 operações e 5 flags, descrita em VHDL com código exclusivamente concorrente e interfaces `std_logic`/`std_logic_vector`, como exigido. Compartilhar um único somador/subtrator entre ADDC, ADD, SUBC e SUB, com inversão do operando B e do carry de entrada, reduziu a lógica e simplificou o cálculo das flags de carry/borrow, *half carry/borrow* e overflow.

Os resultados da simulação concordaram com os valores esperados em todos os casos, inclusive nos seis exemplos de ADD da especificação. Como trabalho futuro, a ALU pode ser integrada a um caminho de dados com registradores de flags, ou ter o desempenho (atraso de propagação e uso de elementos lógicos) comparado entre diferentes arquiteturas de somador.

Referências

ASHENDEN, Peter J. **The Designer's Guide to VHDL**. 3. ed. Burlington: Morgan Kaufmann, 2008.

GARCIA, Francisco; MAGALHÃES, Antônio Hamilton. **Aula 3 – Linguagem de descrição de hardware (HDL): unidades fundamentais, classes, tipos e operadores**. [*S. l.: s. n.*], 2024. Slides da disciplina Sistemas Reconfiguráveis, Engenharia de Computação. rev. D, 22/02/2024.

GARCIA, Francisco; MAGALHÃES, Antônio Hamilton. **Aula 6 – Outros tipos. Exemplos de uso do código concorrente: unidade lógica e aritmética (ALU)**. [*S. l.: s. n.*], 2025. Slides da disciplina Sistemas Reconfiguráveis, Engenharia de Computação. rev. E, 24/09/2025.

GARCIA, Francisco; MAGALHÃES, Antônio Hamilton. **Especificações para o primeiro projeto**. [*S. l.: s. n.*], 2026. Sistemas Reconfiguráveis – Engenharia de Computação, 2º semestre de 2026. Documento em PDF.

HARRIS, Sarah L.; HARRIS, David. **Digital Design and Computer Architecture**. 2. ed. Waltham: Morgan Kaufmann, 2012.

PEDRONI, Volnei A. **Circuit Design and Simulation with VHDL**. 2. ed. Cambridge: MIT Press, 2010.

APÊNDICE A – Arquivos do projeto

O projeto foi organizado para separar a descrição do circuito, os arquivos de simulação, os dados de verificação e os arquivos usados na documentação. A estrutura entregue deve conter os seguintes itens:

`vhdl/alu.vhd` descrição VHDL da ALU, apresentada no Código 4.1. O arquivo contém a entidade, a arquitetura combinacional, o caminho aritmético compartilhado, a seleção das operações e a geração das flags.

`image/` formas de onda exportadas da simulação, uma para cada operação, na mesma ordem apresentada no Capítulo 5: `PASS_B.png`, `OR.png`, `XOR.png`, `AND.png`, `ADDC.png`, `ADD.png`, `SUBC.png`, `SUB.png`, `NOT.png`, `SRA.png`, `SRL.png`, `SLL.png`, `RRC.png`, `RLC.png`, `RR.png` e `RL.png`.

`tabelas/resultados_esperados.md` consolidação dos casos de teste, com entradas, resultados e flags esperadas para as 16 operações.

`tabelas/TP1_<operação>_tabela.md` tabelas individuais dos resultados esperados por operação.

`capitulos/tabela_testes.tex` versão LaTeX das tabelas de teste, usada no Capítulo 5.

`Dissertação.tex` , `config/` e `capitulos/` arquivos principais da documentação, incluindo o preâmbulo, as configurações, os capítulos e este apêndice.